

INFERENCE ENGINEERING

Assignment Report

1. Why is inference memory-bandwidth-bound important?

Inference is often memory-bandwidth-bound because a GPU must continuously move a large amount of model data, including weights and intermediate data, between memory and the compute units.

For a large language model (LLM), the model weights can be very large, and during inference the GPU repeatedly reads these weights to calculate the next token.

If the GPU can perform calculations faster than the required data can be fetched from VRAM, the compute units remain underutilized while waiting for memory. Therefore, memory bandwidth can become the limiting factor rather than raw compute capability.

Example

Consider a GPU with approximately 1 TB/s of memory bandwidth and very high compute capability. If an inference workload requires a large amount of data to be read from VRAM, the performance may be limited by how quickly that data can be transferred rather than by the speed of arithmetic operations.

Why this matters to companies

- Higher memory bandwidth can potentially improve inference speed.
- Faster inference reduces response latency.
- More requests can be served per second.
- Better memory utilization can improve overall GPU utilization.
- Improved efficiency can reduce the cost per generated token.

2. How do we calculate the overall memory required to store a model? How are LLMs represented when stored?

The basic approximation for model-weight memory is:

$$\text{Model Memory} \approx \text{Number of Parameters} \times \text{Bytes per Parameter}$$

Bytes per parameter

Precision	Bytes per Parameter
FP32	4 bytes
FP16 / BF16	2 bytes
INT8	1 byte
INT4	0.5 byte

Example: 7-billion-parameter model

For a model containing 7 billion parameters:

$$\text{FP16 Memory} = 7B \times 2 = 14 \text{ GB}$$

$$\text{INT4 Memory} = 7B \times 0.5 = 3.5 \text{ GB}$$

Therefore:

- 7B FP16 model → approximately 14 GB
- 7B INT4 model → approximately 3.5 GB

The model weights alone require approximately 14 GB in FP16 or 3.5 GB in INT4.

In a real inference system, additional VRAM is required for the KV cache, activations, temporary buffers, and framework/runtime overhead.

How LLMs are represented when stored

When stored, an LLM is primarily represented as numerical tensors containing its learned weights.

The model package also requires supporting information such as:

- Model architecture and configuration
- Tokenizer information
- Quantization-related metadata, where applicable

3. Explain each layer of inference and its importance to an institution/company

An inference system can be understood as a stack of software and infrastructure components.

Each layer has a different responsibility, and together they allow an organization to serve AI models efficiently, reliably, and at scale.

3.1 Runtime

An inference runtime is the execution environment responsible for loading a trained AI model and running inputs through it to generate outputs.

For LLMs, the runtime is optimized to maximize GPU utilization, minimize response latency, and reduce hosting cost.

CUDA

CUDA is NVIDIA's GPU computing platform. It enables software and AI frameworks to use NVIDIA GPUs for fast parallel computation.

PyTorch

PyTorch is a machine-learning framework used to build, train, and run neural-network models.

It can use CUDA to execute computations on NVIDIA GPUs.

vLLM

vLLM is an LLM inference and serving engine that improves serving efficiency through techniques such as continuous batching and efficient KV-cache management.

SGLang

SGLang is an LLM programming and serving framework designed to efficiently run LLM applications, particularly complex generation and structured-output workflows.

TensorRT-LLM

TensorRT-LLM is NVIDIA's optimized inference software for LLMs.

It focuses on inference speed, memory efficiency, and GPU utilization on NVIDIA hardware.

Importance to organizations

In organizational deployments, the runtime layer is important because it directly affects how efficiently expensive GPU resources are used and how quickly users receive model responses.

3.2 Infrastructure

Infrastructure is the collection of hardware and software used to deploy, run, manage, and scale AI applications and models.

Kubernetes

Kubernetes manages and orchestrates containers across multiple machines.

It can:

- Deploy applications
- Restart failed applications
- Manage multiple servers
- Scale applications
- Allocate workloads to available machines

Autoscaler

An autoscaler automatically increases or decreases the amount of running resources according to demand.

This helps:

- Handle sudden traffic
- Avoid overloading servers
- Reduce unnecessary cloud costs

Load Balancer

A load balancer distributes incoming requests among multiple available servers.

Its primary purpose is to prevent a single server from becoming overloaded while other servers remain underutilized.

Capacity Pools

A capacity pool is a group of computing resources kept available for workloads.

In AI inference, a pool can consist of a group of GPU-enabled machines.

Relationship between the components

A useful distinction is that a load balancer decides which available server receives a request, whereas an autoscaler decides how much computing capacity should be available.

Together, they help an inference service handle changing demand efficiently.

4. GPU comparison: RTX 4090 and NVIDIA A100 80GB

The following comparison uses the specifications and rental-cost examples provided in the assignment.

Rental prices can vary by provider, region, and pricing model.

Specification	RTX 4090	A100 80GB
VRAM	24 GB GDDR6X	80 GB HBM2e
Memory Bandwidth	1008 GB/s	≈ 2.0 TB/s
Rental Cost	$\approx \$0.16/\text{hour}$	$\approx \$1.43/\text{hour}$
Architecture	Ada Lovelace	Ampere

4.1 RTX 4090 – theoretical model capacity

16-bit:

$$24 \text{ GB} \div 2 \text{ bytes} \approx 12B \text{ parameters}$$

Therefore, the theoretical weight-only capacity is approximately a 12B-parameter FP16 model.

4-bit:

$$24 \text{ GB} \div 0.5 \text{ byte} \approx 48B \text{ parameters}$$

Therefore, the theoretical weight-only capacity is approximately a 48B-parameter INT4 model.

4.2 A100 80GB – theoretical model capacity

16-bit:

$$80\text{ GB} \div 2\text{ bytes} \approx 40\text{B parameters}$$

Therefore, the theoretical weight-only capacity is approximately a 40B-parameter FP16 model.

4-bit:

$$80\text{ GB} \div 0.5\text{ byte} \approx 160\text{B parameters}$$

Therefore, the theoretical weight-only capacity is approximately a 160B-parameter INT4 model.

Important practical consideration

These are theoretical weight-only estimates.

Practical inference capacity is lower because VRAM must also accommodate:

- KV cache
- Activations
- Runtime buffers
- Framework overhead

5. Serving frameworks: Ollama and LMDeploy

5.1 Ollama

Ollama is a tool for easily running and serving LLMs locally on Windows, macOS, and Linux. It provides a simple command-line interface and API and supports OpenAI-compatible APIs.

What it is good for

- Running LLMs on a personal computer or workstation.
- Local development and testing.
- Quickly downloading and running models.
- Building small AI applications.
- Simple API integration.

When to choose Ollama

Ollama is a suitable choice when simplicity and local usage are more important than maximum production throughput.

For example, a developer who wants to experiment with a Qwen or Llama model on a laptop can use Ollama.

For a large production service with many users, a high-performance serving framework such as vLLM, SGLang, or TensorRT-LLM may be more appropriate.

5.2 LMDeploy

LMDeploy is a toolkit for compressing, deploying, and serving LLMs and vision-language models (VLMs).

It provides two main inference engines:

- TurboMind
- PyTorch

What it is good for

- LLM and VLM inference.
- Production model serving.
- Model quantization.
- OpenAI-compatible APIs.
- Multi-GPU deployment.
- High-performance inference.

LMDeploy supports features such as:

- AWQ/GPTQ quantization
- INT4/INT8 KV cache
- LoRA serving
- Structured output
- Multi-node deployment

TurboMind

TurboMind focuses on high inference performance and optimization.

PyTorch Engine

The PyTorch Engine uses a Python-based implementation that makes development and adding model support easier.

When to choose LMDeploy

LMDeploy may be preferred when:

- Strong quantization support is required.
- Both LLMs and VLMs need to be deployed.
- TurboMind performance optimizations are useful.
- LoRA serving or multi-node deployment is required.
- The model is particularly well supported by LMDeploy.

6. NVIDIA SMI: GPU monitoring and understanding GPU commands

The command `nvidia-smi` is an NVIDIA command-line utility used to inspect and monitor NVIDIA GPU hardware and resource usage.

The exact values shown by the command depend on the GPU machine being inspected.

Example command

```
nvidia-smi
```

Representative output

```
nvidia-smi

GPU: NVIDIA A100
Memory-Usage: 12000 MiB / 80 GB
GPU-Util: 45%
Temperature: 55 C
Power Usage: 180 W
```

Important fields in the output**1. GPU Name**

Shows the NVIDIA GPU model installed in the system, such as an A100.

2. VRAM / Memory Usage

Shows how much GPU memory is currently being used compared with the total available VRAM.

3. GPU Utilization

Indicates how busy the GPU compute resources are at that moment.

4. Temperature

Shows the current GPU temperature and helps identify thermal conditions.

5. Power Usage

Shows the current electrical power being consumed by the GPU.

6. Driver Version

Identifies the installed NVIDIA driver version.

7. CUDA Version

Shows the CUDA version reported as supported by the installed driver.

8. Processes

Lists processes that are currently using the GPU, when available.

How the command works conceptually

The command works conceptually through the following steps:

1. The user executes `nvidia-smi` in the terminal.
2. The command communicates with the NVIDIA driver.
3. The driver obtains status information from the GPU.
4. The collected information is returned to `nvidia-smi`.
5. The utility formats and displays the information in the terminal.

Importance in inference engineering

Therefore, `nvidia-smi` is a command-line utility used to inspect and monitor NVIDIA GPU hardware and resource usage.

It is useful when checking:

- GPU availability
- VRAM usage
- GPU utilization
- Temperature
- Power consumption
- Active GPU workloads

This makes it particularly useful for monitoring AI inference workloads.

Conclusion

Inference engineering involves more than simply running a trained model.

Efficient AI serving requires understanding memory bandwidth, model memory requirements, inference runtimes, GPU hardware, infrastructure, serving frameworks, and operational tooling.

Frameworks such as vLLM, SGLang, TensorRT-LLM, Ollama, and LMDeploy address different deployment requirements.

Infrastructure components such as Kubernetes, autoscalers, and load balancers help organizations operate these systems reliably at scale.

Finally, GPU monitoring tools such as `nvidia-smi` provide practical visibility into the resources used by inference workloads.